Deployment Submission: Gait Classification API

Efe Ozbal - S5996821
Sigurdur Haukur Birgisson - S5982960
Tijje Steeman - S5333644
Will Meeus - S5863708

May 31, 2026

1 What Is Implemented

The project, as discussed with the TA, is in a deployed state. The repository is public and can be opened directly at `https://github.com/ponderingpenguins/applied-machine-learning`. It is hosted live at `https://gait-authentication.sigurdurhaukur.com` with a FastAPI backend and an interactive web interface. The API documentation is available at `https://gait-authentication.sigurdurhaukur.com/docs`. We reccomend trying the live deployment to see the full functionality using any smartphone with a gyroscope and accelerometer (most modern smartphones have these sensors). The web interface allows you to select a model, record your gait data in real time, and see the authentication results with visual feedback.

In the current state, it is a deep learning-based gait authentication system using IMU sensor data. The current implementation includes a training pipeline, a deployed FastAPI API, and a web interface for inspecting and using the models. The root `README.md` documents how to install dependencies and launch the API locally with `uvicorn` or Docker Compose.

Core ML Pipeline

- Dual model architectures (LSTM and Transformer) for gait classification, with embedding-based inference.
- K-fold cross-validation training pipeline with configurable preprocessing (Butterworth, Kalman, FFT filters).
- Centroid-based classification with embedding extraction.
- Automatic model storage and retrieval via Hugging Face Hub.

Web API & Deployment

- FastAPI backend serving inference endpoints (`/models/{model_type}/encode-recording` and `/models/{model_type}/authenticate`).
- Interactive web UI with model selection, real-time sensor input, and result visualization.
- Recent improvements: better UI/homepage, micro-interactions, green/red result screens, timer with audio feedback.
- Docker Compose deployment configuration.

Data Pipeline

- Signal preprocessing with sliding window segmentation.
- StandardScaler normalization (participant-aware).
- FFT-based feature selection.
- Support for raw IMU data (accel x/y/z, gyro x/y/z).

Development & Quality

- Pre-commit hooks with `flake8` linting.
- Code formatted with `black`.
- Dependency management via `uv` package manager.
- Multiple utility scripts (training curves plotting, centroid computation, HF model management).

2 Validation Evidence

We evaluate all three methods on a held-out test set of 18 participants (15% of the dataset) comprising 5,454 gait windows. Test participants have between 117 and 639 windows each (mean 303, std 127), reflecting natural variation in data availability across subjects. We use equal error rate (EER) as the evaluation metric. EER reports the error rate at the threshold where false acceptance rate and false rejection rate are equal, and is standard for biometric verification systems. A random baseline achieves 50% EER. Lower is better.

Method	EER
Transformer	7.68%
LSTM	16.90%
FFT + Centroid	49.54%

Table 1: Equal error rates across three gait classification methods.

The Transformer model substantially outperforms both baselines. The LSTM achieves moderate improvement over hand-crafted FFT features, which perform near chance level. This progression reflects the ability of learned embeddings to capture discriminative gait patterns that frequency-domain features do not.

An EER of 7.68% translates to approximately 8 errors per 100 authentication requests. In practice, the decision threshold can be adjusted based on application requirements. Raising the threshold reduces false acceptances (impostors) at the cost of increasing false rejections (legitimate users), and vice versa. For this project, we maintained the balanced threshold where false positive and false negative rates are equal.

3 API Documentation

The API is documented through FastAPI's interactive interface at `https://gait-authentication.sigurdurhaukur.com/docs` (or `http://localhost:8050/docs` if running locally). The screenshot below shows the endpoint groups and the Pydantic models that define the request and response shapes.

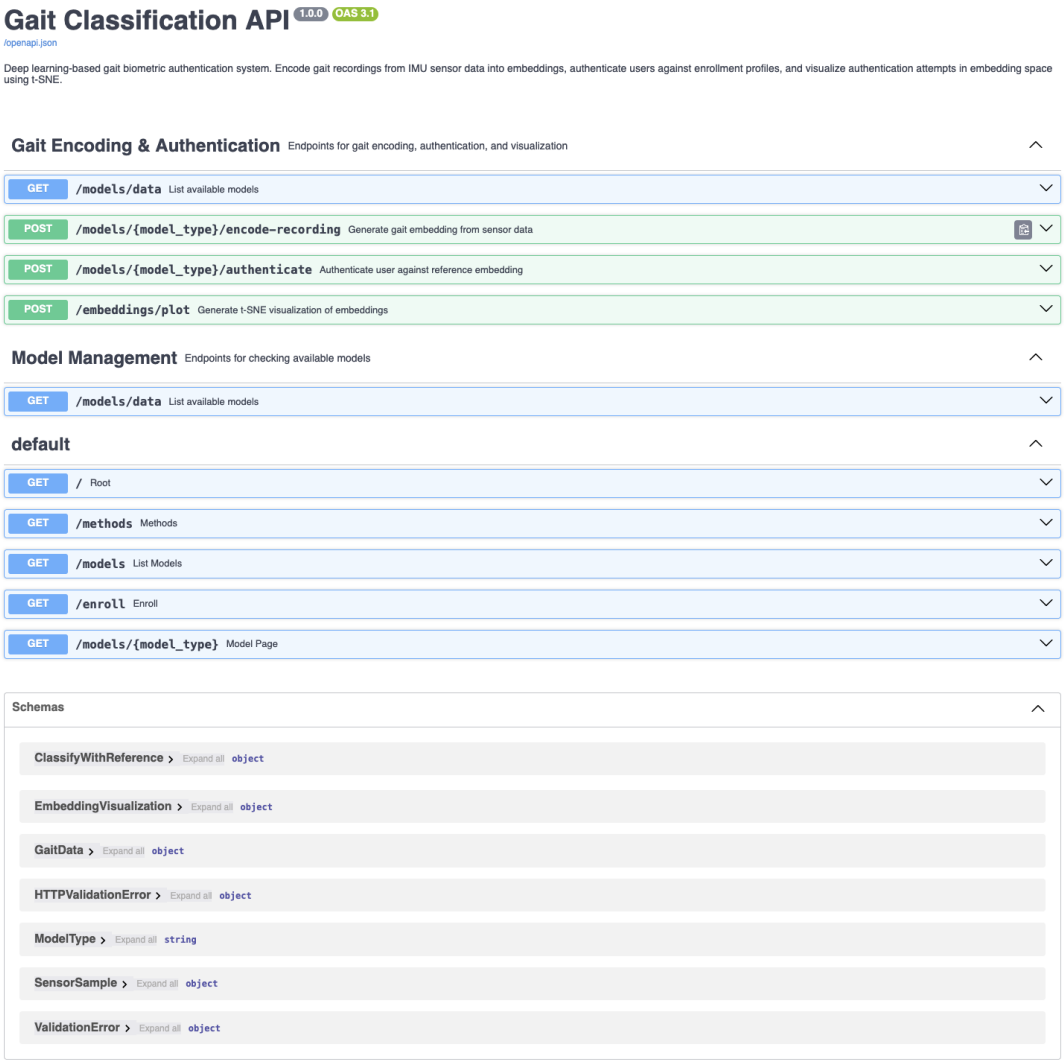

Figure 1: FastAPI documentation showing the endpoints and schema definitions.

The main request types are defined by the following Pydantic models:

- `GaitData`: a list of Inertial Measurement Unit (IMU) sensor samples with accelerometer

and gyroscope values.

- `ClassifyWithReference`: gait samples plus a reference embedding for authentication.
- `EmbeddingVisualization`: a reference embedding and authentication history for plotting.

The main response endpoints are:

- `/models/{model_type}/encode-recording`: returns an `embedding` vector.
- `/models/{model_type}/authenticate`: returns a match decision, distance, threshold, confidence, samples processed, and the computed embedding.
- `/embeddings/plot`: returns a t-SNE plot of the reference embedding and authentication history for visualization.

The request/response screenshot below shows a successful API call made with `curl` and the returned JSON response. In this example, we sent IMU data (gyroscope and accelerometer readings) to the `/models/lstm/encode-recording` endpoint to embed the gait data, and we received the expected embedding vector in the response, then we sent another request to the `/models/lstm/authenticate` endpoint with IMU data of a different recording and the previous embedding as a reference (the model compares the similarity of the new embedding with the reference embedding to determine if they match). The response indicates that the two recordings do match, with a distance of 0.4 (just below the threshold of 0.5) and a confidence score of 0.6, meaning the model is reasonably confident that the two recordings are from the same person. The response also includes the number of samples processed and the computed embedding for the new recording. This demonstrates that the API is functioning correctly and can process gait data to perform authentication based on the learned embeddings.


(base) deployment git:(dev/haukur) curl -X 'POST' \
'https://gait-authentication.sigurdurhaukur.com/models/lstm/encode-recording' \
-H 'accept: application/json' \
-H 'Content-Type: application/json' \
-d '{
"samples": [
{
"acc_x": 0.5,
"acc_y": -0.2,
"acc_z": 9.8,
"gyr_x": 0.1,
"gyr_y": 0.05,
"gyr_z": -0.15
},
{
"acc_x": 0.6,
"acc_y": -0.1,
"acc_z": 9.7,
"gyr_x": 0.12,
"gyr_y": 0.07,
"gyr_z": -0.12
}
]
}'
{"embedding":[-0.04354763403534889,-0.12422680979549408,-0.06274624168872833,-0.12338657677173615,0.03975304961204529,-0.11925270408391953,0.05486937984241806,-0.10124969482421875,-0.01459924597293138
deployment git:(dev/haukur) curl -X 'POST' \
'https://gait-authentication.sigurdurhaukur.com/models/lstm/authenticate' \
-H 'accept: application/json' \
-H 'Content-Type: application/json' \
-d '{
"reference_embedding":[-0.04354763403534889,-0.12422680979549408,-0.06274624168872833,-0.12338657677173615,0.03975304961204529,-0.11925270408391953,0.05486937984241806,-0.10124969482421875,-0.01459924597293138
"samples": [
{
"acc_x": 0.5,
"acc_y": -0.2,
"acc_z": 9.8,
"gyr_x": 0.1,
"gyr_y": 0.05,
"gyr_z": -0.15
}
]
}'
{"is_match":true,"distance":0.4195646345615387,"threshold":0.5,"confidence":0.5804353654384613,"samples_processed":1,"embedding":[-0.004766448865575956,-0.12723630666732788,-0.05727555602788925,-0.11881615357398987,0.06940591335296631,-0.17
Figure 2: Example API request and successful response.

4 Contributions

Sigurdur Haukur Birgisson

- ML pipeline foundation: triplet mining, online triplet loss, and k-fold training.
- Data pipeline: preprocessing filters, windowing, and standardization.
- Model architectures: LSTM and Transformer implementations.
- Backend/frontend integration: FastAPI routes, web UI, and model inference.
- DevOps: Docker containerization and Hugging Face integration.

Peter Meeus

- K-fold cross-validation implementation.
- Evaluation metrics: FAR, FRR, and EER computation for unseen participants.
- Model parameter tuning and finetuning workflow.
- CosFace loss integration, in collaboration with Tijje.
- Training visualization and loss/embedding analysis.
- Dataset handling fixes and preprocessing optimizations.
- DevOps: pre-commit hooks, code formatting, and dependency management.

Efe

- Initial FastAPI endpoint design and setup.
- Template system and HTML frontend pages.
- API webpage and user interface development.
- Feature separation: model pages, user enrollment, and classification logic.
- API testing framework.
- Filter experimentation and visualization.

Tijje

- Custom linear layer for CosFace loss.
- CosFace loss function implementation and integration.
- Alternative loss function support alongside triplet loss.
- Model parameter tuning code and setup support.

5 AI Use

We used Copilot minimally for boilerplate code generation, LaTeX formatting suggestions, and an unsuccessful attempt to help resolve a merge conflict. Copilot was involved in commits 6e4e38c, c5edb3e, 1a87b66, e3dbae3, 3f6da29, and f03046b.